Open in Colab

Python List Objects

To modify a tuple in Python you must create a new tuple. This is because tuples are readonly collections. If you want a **mutable** collection of objects you should use *lists*.

To create a list in Python, write some Python objects between **brackets** seperating them with `,`.

In [ ]:
```
['hello', 'world', 1, None, 1]
```

Like tuples, Python also allows one more `,` at the end of the list. This extra `,` does not have an effect. The list created will be the same.

In [ ]:
```
['hello', 'world', 1, None, 1, ]
```

Unlike tuples, you can create a list with single object without using extra `,` at the end.

In [ ]:
```
['hello']
```

An empty list can be created by:

In [ ]:
```
[]
```

Since list is one Python object then you can assign it to a variable.

In [ ]:
```
h = ['hello']
w = ['world', 1]
```

Operations on Lists

Any operation on tuples are also aplicable to the lists in the same way.

In [ ]:
```
len(w)
```

In [ ]:
```
'world' in w
```

In [ ]:
```
w.count('hello')
```

In [ ]:
```
h + w
```

In [ ]:
```
(h + w) * 3
```

You can also nest tuples and lists like tuples. Indeed you can nest tuples to lists and lists to tuples.

In [ ]:
```
a = ['one', 2, (3, [4, 5])]
```

You can also access the items and ranges of a list like a tuple.

In [ ]:
```
a[-1][1][0:-1]
```

Modifying The Lists

In addition to tuples, lists are **mutable**.

Changing An Item of The List

You can change any item by assigning it a different object.

In [ ]:
```
a[-1] = "last"
a
```

You can also change a range of items from a list.

In [ ]:
```
a[::2] = [1,1]
a
```

Removing An Item From The List

To remove the object `obj` from a list use `.remove(obj)` method.

In [ ]:
```
a.remove(1)
a
```

You can delete an item from a list by `del` keyword.

In [ ]:
```
del a[0]
```

Now the first item in the list is removed.

In [ ]:
```
a
```

To rewmove all the items from a list use `.clear()` method.

In [ ]:
```
a.clear()
a
```

Adding New Items To The List

To add an object `obj` at the end of the list use `.append(obj)` method.

In [ ]:
```
a.append(8)
a
```

To add multiple objects in a single method use `.extend(iterable)` method with a list `iterable` (or any iterable):

In [ ]:
```
a.extend([19,10,11])
a
```

To add an object `obj` at an arbitrary `index` shifting the remaining, use `.insert(index,obj)` method.

In [ ]:
```
a.insert(20,-10)
a
```

Here since the index 20 is invalid Python assumed it as last.

Reordering The Lists

In [ ]:
```
a.sort()
a
```

In [ ]:
```
a.reverse()
a
```

Making A Copy of The List

If you write:

In [ ]:
```
b = a
```

Then Python gives just another name to the same list object. If you modify `b` here then `a` is also modified because they are essentially the same. For example:

In [ ]:
```
b[0] = None
```

In [ ]:
```
a
```

In [ ]:
```
b
```

To make a copy of a list use `.copy()` method.

In [ ]:
```
b = a.copy()
```

Now `a` and `b` are different list objects.

In [ ]:
```
del a[0]
```

In [ ]:
```
a
```

In [ ]:
```
b
```

Also range operations create a new list. So writing:

In [ ]:
```
b = a[:]
```

will also make a copy of the list `a`.

Python Set Objects

Python sets are **unordered, mutable, non-repeating** collections. Like the sets in mathematics, sets are created by braces.represented by braces.

To create a set in Python, write some *(hashable)* Python objects between braces seperating them with `,`.

Some examples to the hashable objects are integers, strings, floating point numbers, `True`, `False`, `None` and tuples. But lists, sets and dictionaries are not hashable. So you can not nest sets.

In [ ]:
```
s = {'hello', 'world', 1, None, 1,(1,2)}
s
```

Even if you write the same item multiple times in a set items can not repeat.

Sets are **un-ordered**, meaning the order of items are not preserved. Therefore unlike lists and tuples you can not access itesm by their index.

Empty Set

Warning: The empty set is created by `set()` not `{}`

Testing The Membership

With `in` keyword you can test if an object is in a set or not.

In [ ]:
```
(1,2) in s
```

Adding New Items

You can add new hashable items to a set. If it is already in the set then it is ignored.

In [ ]:
```
s.add(3.14)
s
```

You can also use `.update(iterable)` method to add multiple items using a set `iterable` (or any iterable).

In [ ]:
```
s.update({1,2,3})
s
```

Removing An Item From The Set

`.discard(obj)` and `.remove(obj)` method are both used for removing an object `obj` from the set. The difference is `.remove(obj)` yields `ValueError` if `obj` is not in the set. While `.discard(obj)` simply ignores if it does not exist.

In [ ]:
```
s.remove(3.14)
s.discard(3.14)
s
```

Matematical Operations On Sets

You can do mathematical set operations on sets with:

- `.union(iterable)`
- `.intersection(iterable)`
- `.difference(iterable)`
- `.symmetric_difference(iterable)`
- `.issubset(iterable)`
- `.issuperset(iterable)`
- `.isdisjoint(iterable)`

methods. Where `iterable` is a set, list, tuple or any iterable.

Warning: These methods do not modify the set itself. But they create a new set.

In [ ]:
```
t = {'a','b','c'}
```

In [ ]:
```
s.union(t)
```

In [ ]:
```
s.intersection(t)
```

In [ ]:
```
s.difference(t)
```

In [ ]:
```
s.symmetric_difference(t)
```

In [ ]:
```
s.issubset(t)
```

In [ ]:
```
s.issuperset(t)
```

In [ ]:
```
s.isdisjoint(t)
```

Python Dictionary Objects

Dictionaries are simple mappings that associate some hashable objects to some objects.

To create a dictioanry in Python, write some `key:value` pairs between braces seperating them with `,`. Here `key` must be a hashable Python object like string integer floating point number or a tuple and value a Python object.

In [ ]:
```
d = {
    'name':'Ali',
    'surname':'Cin',
    'age':17,
    'address':{
     'country':'Türkiye',
     'address': 'Antalya',
     },
    'friends':['Ahmet','Mehmet'],
    17: 'ok'
    }
```

Dictionary keys are like set items, must be unique and hashable. But values can be any Python object so they can be nested.

Empty dictionary can be created by `{}`

Accessing The Items Of The Dictionary

In [ ]:
```
d['address']['country']
```

In [ ]:
```
d[17]
```

To test if a `key` exists in a dictionary use `in` keyword.

In [ ]:
```
'job' in d
```

If the `key` does not exist Python will yield a `KeyError`. To prevent this, you can use `.get(key)` method. `.get(key)` method gives `None` if the key is not in the dictionary.

In [ ]:
```
d.get('job')
```

Modifying The Dictionary

You can change the values and also add new `key:value` pairs to a dictionary.

In [ ]:
```
d['age'] = 18
d['gender'] = 'male'
d
```

In [ ]:
```
d.update({'age':19})
d
```

Use `.setdefault(key,value)` to add a `key:value` pair in to the dictionary if the `key` does not exists. If it is already in the dictionary this method gets that value.

In [ ]:
```
d.setdefault('friends',[])
```

To remove a `key:value` pair use `del` keyword.

In [ ]:
```
del d[17]
d
```

The `.clear()` method remove all items from the dictioanry. And the `.copy()` method makes a copy of the dictionary.

Geting The List of Keys, Values And Key-Value Pairs

In [ ]:
```
d.keys()
```

In [ ]:
```
d.values()
```

In [ ]:
```
d.items()
```

Creating New Collections From The Existing Ones

You can make new tuples from other iterables like strings tuples, lists, sets and dictionaries with builtin `tuple()` function.

In [ ]:
```
tuple("abc")
```

In [ ]:
```
tuple((3,7,1))
```

In [ ]:
```
tuple([3,7,1])
```

In [ ]:
```
tuple({3,7,1})
```

In [ ]:
```
tuple({3:9,7:49,1:1})
```

You can make new lists from other iterables like strings tuples, lists, sets and dictionaries with builtin `list()` function.

In [ ]:
```
list((3,7,1))
```

You can make new sets from other iterables like strings tuples, lists, sets and dictionaries with builtin `set()` function.

In [ ]:
```
set((3,7,1))
```

You can make new dictionaries from other iterables like tuples, lists, sets and dictionaries with builtin `dict()` function.

In [ ]:
```
dict(((3,9),(7,49),(1,1)))
```

Comprehensions

In mathematics using the set builder notation $\{x^2 \mid x \in A\}$ you can represent the squares of each element in the set A. Similarly you can use comprehensions in Python to get similar functioanlity by:

In [ ]:
```
{x**2 for x in a}
```

It can also be a list or dictioanry.

In [ ]:
```
[x**2 for x in a]
```

In [ ]:
```
{x:x**2 for x in a}
```

Warning: You can not create a tuple with comprehension.

When using the set builder notation you can also add some conditions like $\{x^2 \mid x \in A, x \equiv 0 \mod 2\}$. You can use comprehensions in Python to get similar functionality by:

In [ ]:
```
{x**2 for x in a if x % 2 == 0}
```